```
  if (! Hash::check($request->password, $request->user()-
  >password)) {
   return back()->withErrors([
    'password' => ['Incorrect Password']
   ]);
  }
  $request->session()->passwordConfirmed();
  return redirect()->intended();
 })->middleware(['auth', 'throttle:6,1']);
```

Let's take a closer look at this path before going on. First, the request's **`password`** field is checked to see if it matches the password of the authorized user. If the password is correct, we must notify Laravel's session that the user has validated their password. The **`passwordConfirmed`** method adds a timestamp to the user's session, which Laravel may use to detect when the user last verified their password. Finally, we may guide the user to their desired location.

Authentication Custom Guards

Using the **`extend`** method on the Auth facade, you may create your own authentication guards. You should make a call to a service provider's **`extend`** method. Because Laravel already has an **`AuthServiceProvider`**, we can paste the following code into it:

```
<?php
namespace App\Providers;
use App\Services\Auth\JwtGuard;
use Illuminate\Contracts\Foundation\Application;
use Illuminate\Foundation\Support\Providers\AuthServiceProvider
as ServiceProvider;
use Illuminate\Support\Facades\Auth;
class AuthServiceProvider extends ServiceProvider
{
 /**
  * Register any application authentication / authorization
  services.
  */
 public function boot(): void
```